CMP-6048A/7009A Advanced Programming

Mathematical Interpreter

Group members:
Charlie Flux
Ali Ansari

School of Computing Sciences, University of East Anglia

Version 2.0

Abstract

This project presents the design and implementation of a mathematical programming language platform featuring an interpreter, a C transpiler and a RISC-V compiler developed in a functional programming paradigm with an accompanying graphical user interface. The language was defined using Backus-Naur form and implemented using a unified abstract syntax tree and semantic model, enabling programs to be executed either by direct interpretation or by transpilation to C, preserving program semantics across backends. The RISC-V compiler produces assembly code compatible with simulators and physical hardware. A key component of the project is the graphical interface, which allows users to write and evaluate programs and interactively visualise functions through plots.

Chapter 1

Introduction

1.1 Project statement

This project aims to implement an interpreter, C transpiler & RISC-V Compiler in F# with an accompanying GUI in C# using WPF. The language will be defined such that both execution backends operate over a common abstract syntax tree and semantic model produced by a shared frontend. Consequently, programs written in the language can be evaluated either by direct interpretation or by transpilation to C, with execution delegated to a standard C compiler while preserving program semantics. A stand-alone RISC-V compiler also exists to translate programs into RISC-V assembly, enabling execution on hardware simulators or physical RISC-V processors. The GUI provides an interactive interface for writing, evaluating and debugging programs, displaying results and visualising runtime errors in a user-friendly manner. The GUI also features plotting functionality to allow the user to visualise mathematical functions directly within the environment, enhancing usability and program analysis.

1.2 Aims and objectives

The primary aim of this project is to develop a mathematical interpreter with support for multiple execution backends. The system operates in three modes: direct interpretation via a tree-walking evaluator, transpilation to C code and compilation to RISC-V assembly. All three backends share a common frontend comprising a lexical analyzer and recursive descent parser that generates a unified Abstract Syntax Tree representation.

The core objectives are:

- Implement a complete language processing pipeline including lexical analysis, syntax analysis and semantic evaluation
- Build a tree-walking interpreter for immediate expression evaluation with support for variables, control flow and user-defined functions
- Develop a C transpiler that generates executable code through a Three Address Code intermediate representation
- Create a graphical user interface in C# with WPF for code editing, program execution and function visualization
- Maintain semantic consistency across all execution modes

To organize project requirements systematically, a MoSCoW analysis was conducted to prioritize functional requirements. Table 1.1 categorizes tasks into four priority levels: **Must have** features form the essential deliverable, **Should have** features enhance the system capability, **Could have** features represent optional extensions, and **Should not have** items define explicit scope boundaries. This prioritization framework guided the

iterative development process, ensuring fundamental functionality was established before implementing advanced features.

Non-functional requirements, including code quality and error handling, are addressed throughout the implementation chapters.

Table 1.1: MoSCoW - Functional Requirements

Priority	Task	Comments
Must	Lexical Analysis	Tokenize numbers (int/float/scientific), identifiers, operators, keywords
	Syntax Analysis	Recursive descent parser with AST generation and correct precedence
	Interpreter	Tree-walking interpreter with symbol table and dynamic typing
	C Transpiler	Generate C code via TAC intermediate representation
	Type System	Type inference with promotion (int, float, complex)
	Error Handling	Meaningful error messages for lexical, syntax, and runtime errors
Should	Control Flow	If-then-else conditionals, for-loops, while-loops with comparison operators
	Functions	User-defined functions with recursion and built-in math functions (sin, cos, sqrt, print, etc.)
	Complex Numbers	Full complex arithmetic with type promotion
	WPF GUI	C# interface with REPL, code editor, output display, symbol table visualiser
	Function Plotting	OxyPlot integration with configurable ranges, interactive zoom/pan, grid lines, asymptote detection
Could	RISC-V Compiler	Assembly generation with stack frames and register allocation
	Extended Features	String type, hyperbolic functions (sinh, cosh, tanh), TAC optimization passes
	GCC Integration	Compile and execute generated C code directly from GUI
Should not	Non-Functional Design	Object-oriented features (classes, methods, inheritance) excluded in favor of functional programming approach
	Automated Tools	Parser generators (lex, yacc) excluded per brief requirements

Chapter 2

Background

To satisfy the requirements of this project, the core of the project was written in F# [Microsoft Learn, F#, 2026]. F# follows the functional programming language paradigm, drawing heavily on the principles of lambda calculus to emphasise immutability, high-order functions and declarative program structure. It's features make it particularly suited to the implementation of language development, where programs are represented and transformed as abstract syntax trees and intermediate representations. Functional programming encourages the use of pure functions and explicit data flow which simplifies the stages of an interpreter, transpiler or compiler where pattern matching and strong static typing are essential when working with algebraic data types during parsing, semantic analysis, optimisation and code generation. The use of algebraic data types allows a single abstract syntax tree to be shared between the interpreter, transpiler and compiler components of the system. This creates a clear logical separation of the front-end, which is responsible for parsing and semantic analysis, and the back-end, which performs program transformation and target specific code generation. A front & back design approach is widely adopted in modern compiler architectures.

This project is conceptually similar to other mathematical computing environments in particularly Julia [JuliaHub, Inc., 2026]. The open-source nature of Julia inspired this project to target the modern open source instruction set of RISC-V. Other solutions such as MATLAB [MathWorks ®, 2026b] do not run natively on RISC-V processors [MathWorks ®, 2026a] however, MATLAB software provides the ability to generate C++ code by the use of a transpiler. The software in this project mirrors this approach and by allowing programs to be transpiled into C code, thereby extending the target platform of our implemented language to any system a compatible C compiler. Offloading the target machine dependency is an attractive solution for newer languages, since implementing and maintaining support for a diverse set of target architectures significantly increases implementation complexity, accumulates technical debt and diverts the scope of a mathematical language. Numerous modern languages adopt this approach to extend platform comparability, notably Nim [Andreas Rumpf, 2026], a statically typed, compiled, multi-paradigm system programming language which translates high level source code into C & C++ code.

The graphical user interface was implemented using Windows Presentation Foundation (WPF) [Microsoft Learn, WPF, 2026] which is a highly documented and supported .NET graphical framework and a informal successor to the popular Visual Basic language [Microsoft Learn, Visual Basic, 2026]. WPF was chosen due to the easy integration into a .NET project, that encompasses both F# and C# code, simplifying the setup overhead of build systems. The GUI build is strictly limited to Windows based operating systems however, open-source alternatives such as Avalonia [AvaloniaUI OÜ, 2026] exist to offer cross-platform .NET graphical interfaces. To add plotting functionality, the GUI used the open-source plotting library OxyPlot [OxyPlot, 2026]. OxyPlot integrates easily into existing .NET projects allowing deep control over plotting functionalities implemented in the Model-View-Controller (MVC) software design pattern.

Chapter 3

Development History

3.1 Sprint 1: Basic expressions and GUI

3.1.1 Grammar in BNF

The first BNF enforced correct arithmetic procedure (BIDMAS) and evaluated expressions in a left associative manner. It also supported unary plus and minus operators, allowing signed and nested numeric operators. The functionality was limited to integer arithmetic and no support for exponentials or other higher order operators.

```
<E>       ::= <T> <Eopt>
<Eopt>    ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
<T>       ::= <NR> <Topt>
<Topt>    ::= "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<NR>      ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")"
```

3.1.2 Basic GUI

Figure 3.1 shows the first implementation of the GUI that was built with WPF in C#. A proof of concept OxyPlot window was added however, there was no link between the input and plotting functionality. Due to simple nature of the GUI, no MVVM principles were followed for this build of the GUI.

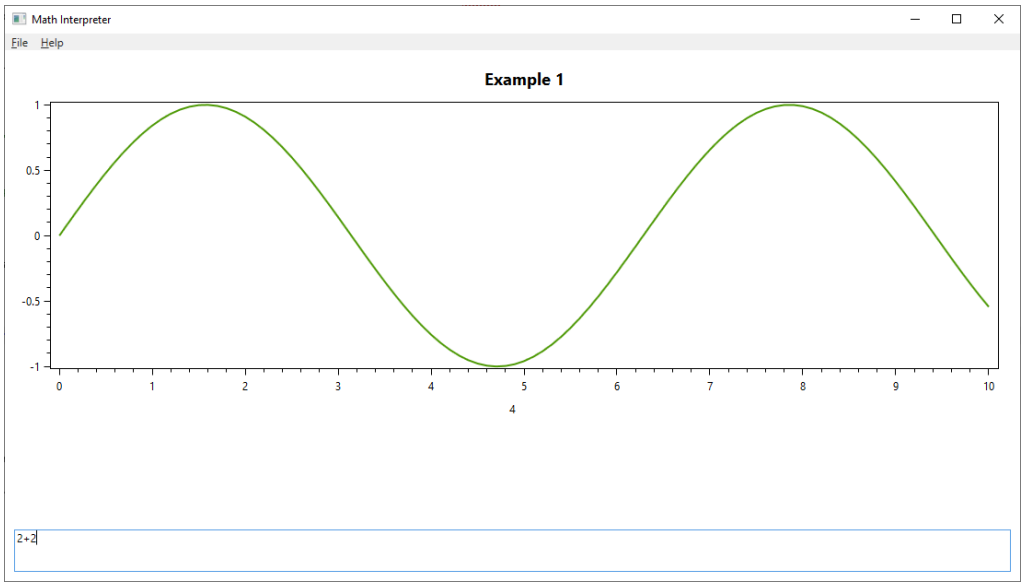

Figure 3.1: Sprint 1 GUI

3.1.3 Testing

A subset of Table B.5 in Appendix B could be referred to from here.

3.2 Sprint 2: Adding powers, modulo and MVVM

3.2.1 BNF

The BNF was updated to include power functionality by the addition of `<P>` & `<Popt>` nonterminal symbols. Furthermore, the modulo % operator was added.

```
<E>         ::= <T> <Eopt>
<Eopt>      ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
<T>         ::= <NR> <Topt>
<Topt>      ::= "%" <NR> <Topt> | "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<P>         ::= <NR> <Popt>
<Popt>      ::= "^" <P> | <empty>
<NR>        ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")"
```

3.2.2 Updated GUI

Visually the GUI stayed the same as in sprint 1; however, the architecture of the GUI implementation was updated to follow MVVM principles, laying the foundations for future builds, as seen in Figure **??**. This involved creating view models for each window and adding the supporting view locators and window services.

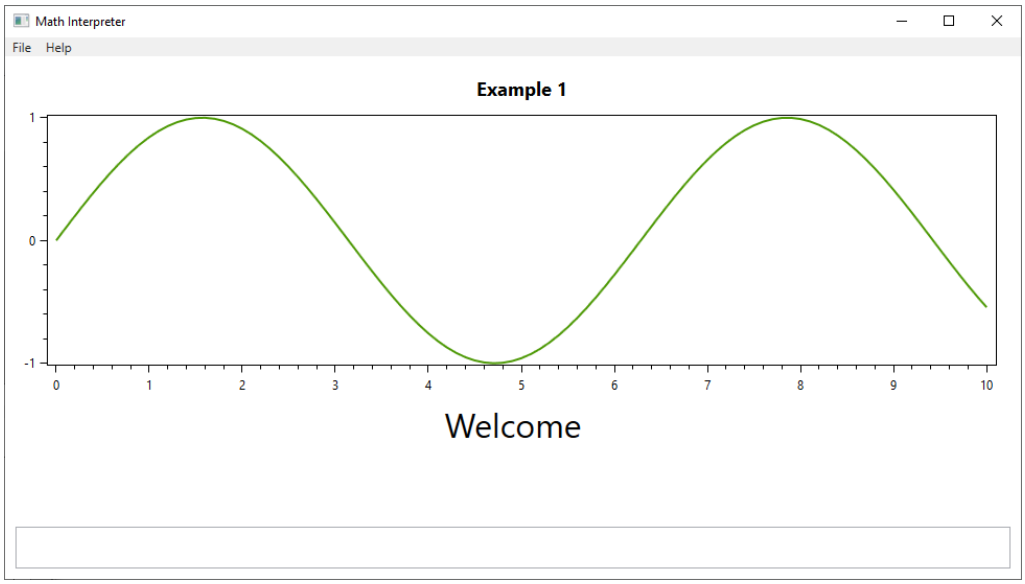

Figure 3.2: Sprint 2 GUI

3.3 Sprint 3: Function Calls, Plotting and GUI Updates

3.3.1 BNF

Function calls consist of `<FCall>`, `<Args>` and `<ArgList>` nonterminal symbols. The parser detects the function name from a list of known functions (e.g. `sin`, `cos`) and recursively parses each argument as expressions and stores them in argument list.

```
<E>         ::= <T> <Eopt>
<Eopt>      ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
```

```
<T>         ::= <NR> <Topt>
Topt>       ::= "%" <NR> <Topt> | "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<P>         ::= <F> <Popt>
<Popt>      ::= "^" <P> | <empty>
<F>         ::= <NR> | <FCall>
<NR>        ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")"
<FCall>     ::= "id" "(" <Args> ")"
<Args>      ::= <E> <ArgList> | <empty>
<ArgList> ::= "," <E> <ArgList> | <empty>
```

3.3.2 Updated GUI

This sprint saw the largest overhaul of the GUI. Plotting functionality was added by implementing a plotting function evaluator in F#, which computes the value of y for each x. Due to the limitation of only integer values being supported, highly detailed plots were not yet available. A message parsing protocol was introduced to manage communication between the GUI and the interpreter, standardising how results are returned. Each message consists of a header, which identifies the data type, and the associated payload. In addition to the GUI's Read–Eval–Print Loop behaviour, an input history scroll viewer was added, mirroring the command history functionality of terminal environments such as Bash [Free Software Foundation, 2024] or the Python REPL [Python Software Foundation, 2024], and enabling users to revisit previous inputs.

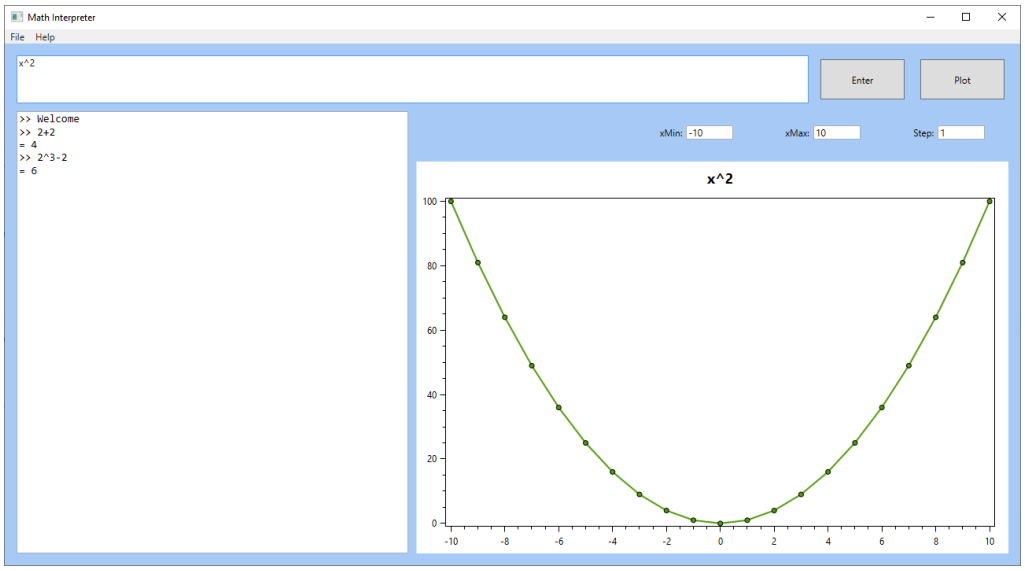

Figure 3.3: Sprint 3 GUI

3.4 Sprint 4: Floating Point Values

3.4.1 BNF

Support for both integer and floating point operations was subsequently implemented. This change did not require modification of the numeric agnostic BNF; instead, the parsing and evaluation stages were updated to introduce a clear separation between integer and floating point values. A custom `NumericValue` type was created to encapsulate either an integer or floating point value, with arithmetic being handled via dedicated helper functions.

```
<E>         ::= <T> <Eopt>
<Eopt>      ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
```

```
<T>         ::= <NR> <Topt>
<Topt>      ::= "%" <NR> <Topt> | "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<P>         ::= <F> <Popt>
<Popt>      ::= "^" <P> | <empty>
<F>         ::= <NR> | <FCall>
<NR>        ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")"
<FCall>     ::= "id" "(" <Args> ")"
<Args>      ::= <E> <ArgList> | <empty>
<ArgList> ::= "," <E> <ArgList> | <empty>
```

NumericValue Arithmetic

- If both operands are integers, the result is an integer.
- Otherwise, operands are promoted to floating point, and the result is a float.

Pseudo-code for multiplication can be expressed as:

```
function mulNums(a: NumericValue, b: NumericValue):
    if a and b are integers:
        return integer multiplication
    else:
        convert a and b to float
        return floating-point multiplication
```

3.4.2 Updated GUI

The GUI saw little change. The most notable change was support for fractional step values to allow for more detailed plots as seen in Figure 3.7

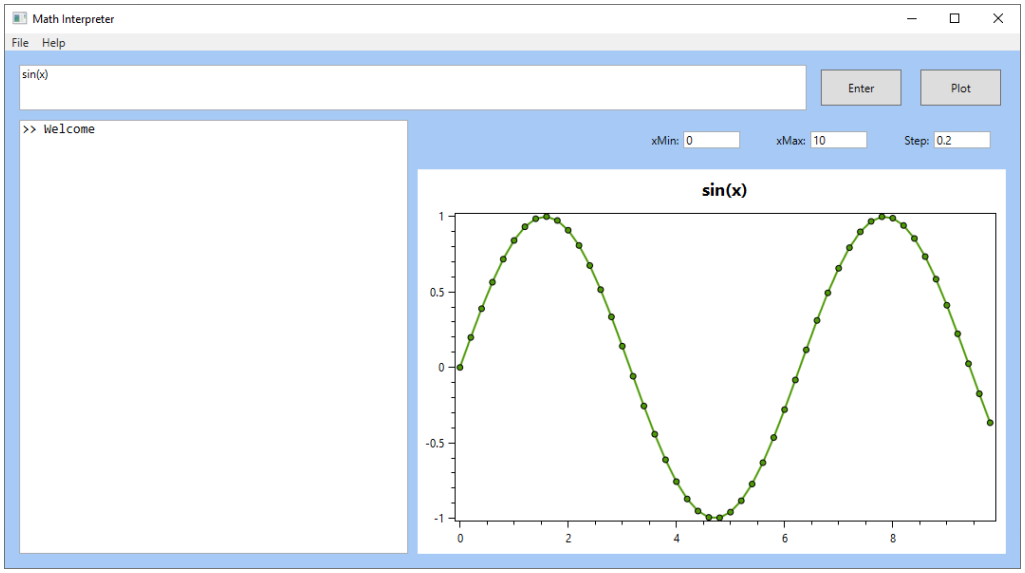

Figure 3.4: Sprint 4 GUI

3.5 Sprint 5: Assignment Operator and Symbol Tables

3.5.1 BNF

Sprint 5 focused on implementing the assignment operator. To achieve this, the statement nonterminal symbol (`<S>`) was added to the BNF, allowing an optional assignment of an

expression to an identifier (`Id = <E>`). This enabled the interpreter to assign and evaluate variables using a mutable map that stores identifier–value pairs, which are retrieved during the evaluation stage. A persistent symbol table was maintained across REPL iterations, ensuring that variable assignments remained available for subsequent expressions.

```
<S>         ::= Id "=" <E> | <E>
<E>         ::= <T> <Eopt>
<Eopt>      ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
<T>         ::= <NR> <Topt>
<Topt>      ::= "%" <NR> <Topt> | "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<P>         ::= <F> <Popt>
<Popt>      ::= "^" <P> | <empty>
<F>         ::= <NR> | <FCall>
<NR>        ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")"
<FCall>     ::= Id "(" <Args> ")"
<Args>      ::= <E> <ArgList> | <empty>
<ArgList> ::= "," <E> <ArgList> | <empty>
```

3.5.2 Updated GUI

The updates in sprint 5 solely focused on BNF changes therefore, no updates to the GUI were made.

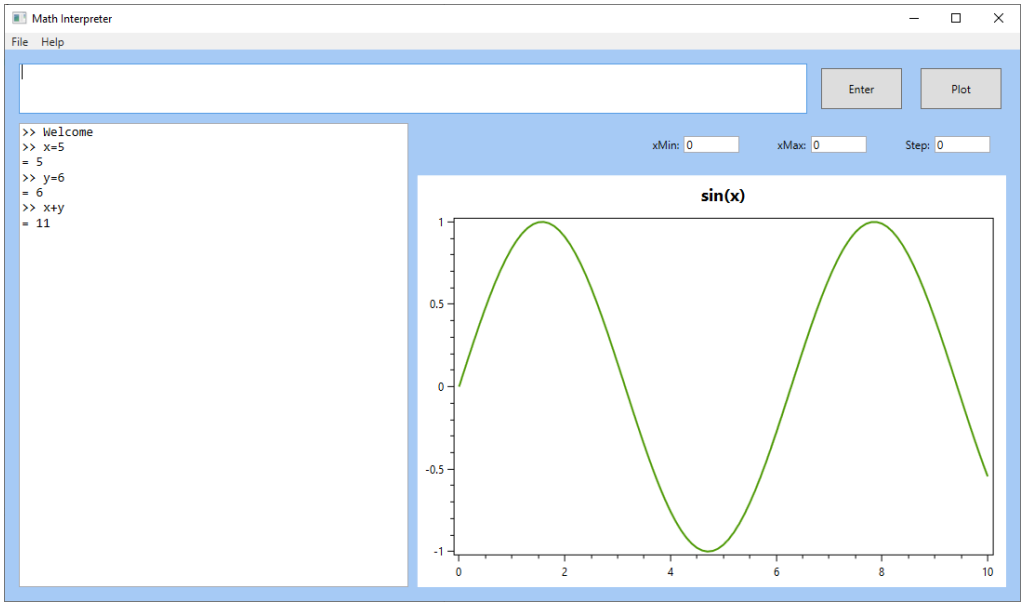

Figure 3.5: Sprint 5 GUI

3.6 Sprint 6: Programs and Control Flow

3.6.1 BNF

Sprint 6 implemented the traditional program structure to the language. A program is list of semi-colon separated statements, allowing multiple line input to the interpreter as seen in Figure 3.6. This change enables the implementation of more complex control flow including `if`, `for` and `while` loops. Using this new feature, a simple `if then else` statement was implemented to allow for simple control flow. An equality comparison operator is available to enable conditional branching. The BNF below documents these changes with dedicated `<Comp>` and `IfStmt` nonterminal symbols.

```
<Prog>     ::= <S> <Progopt>
<Progopt>  ::= ";" <S> <Progopt> | <empty>
<S>        ::= Id "=" <Comp> | <Comp>
<Comp>     ::= <E> | <E> "==" <E>
<E>        ::= <T> <Eopt>
<Eopt>     ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
<T>        ::= <P> <Topt>
<Topt>     ::= "%" <NR> <Topt> | "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<P>        ::= <F> <Popt>
<Popt>     ::= "^" <P> | <empty>
<F>         ::= <IfStmt> | <NR> | <FCall>
<IfStmt>    ::= "if" "(" <Comp> ")" "then" "{" <Prog> "}" ("else" "{" <Prog> "}")?
<NR>       ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")" | Id
<FCall>    ::= Id "(" <Args> ")"
<Args>     ::= <Comp> <ArgList> | <empty>
<ArgList> ::= "," <Comp> <ArgList> | <empty>
```

If Then Else Statement

An example If Then Else statement is shown below:

```
// if(comparison) then expr else expr

x = 0;
if(2 == 2) then {
    x = 5;
} else {
    x = 10;
}
Result: x = 5
```

3.6.2 Updated GUI

Sprint 6 included adding support for plotting markers showing the values that were calculated on a plot. Figure 3.6 shows a plot of $sin(x)$ using the markers. Furthermore, a symbol table visualiser was implemented showing variable name, value and data type.

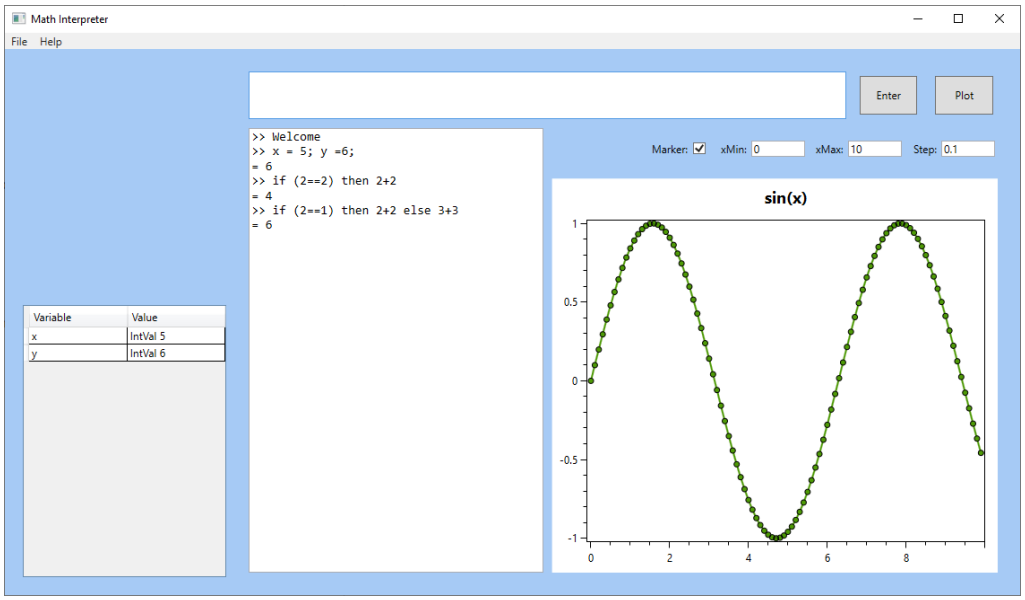

Figure 3.6: Sprint 6 GUI

3.7 Sprint 7: Advanced Control Flow and Plot Scaling

3.7.1 BNF

Sprint 7 extended the control flow capabilities introduced in the previous sprint to include `for` and `while` loops. These high-level constructs are semantically similar and are internally represented as sequences of conditional checks and jumps, which simplifies the lowering of the Intermediate Representation (IR) to Three-Address Code (TAC). Conditional loops require more comprehensive comparison operators: `< >`.

```
<Prog>      ::= <S> <Progopt>
<Progopt>   ::= ";" <S> <Progopt> | <empty>
<S>         ::= Id "=" <Comp> | <Comp>
<Comp>      ::= <E> | <E> "==" <E> | <E> "<" <E> | <E> ">" <E>
<E>         ::= <T> <Eopt>
<Eopt>      ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
<T>         ::= <P> <Topt>
<Topt>      ::= "%" <NR> <Topt> | "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<P>         ::= <F> <Popt>
<Popt>      ::= "^" <P> | <empty>
<F>         ::= <IfStmt> | <ForLoop> | <WhileLoop> | <NR> | <FCall>
<IfStmt>    ::= "if" "(" <Comp> ")" "then" "{" <Prog> "}" ("else" "{" <Prog> "}")?
<ForLoop>   ::= "for" "(" Id "=" <Comp> "to" <Comp> ")" "do" "{" <Prog> "}"
<WhileLoop> ::= "while" "(" <Comp> ")" "do" "{" <Prog> "}"
<NR>        ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")" | Id
<FCall>     ::= Id "(" <Args> ")"
<Args>      ::= <Comp> <ArgList> | <empty>
<ArgList>   ::= "," <Comp> <ArgList> | <empty>
```

3.7.2 Updated GUI

The GUI saw a major upgrade by the introduction of automatic plot scaling. As the user adjusted the x and y axis, the plotting engine dynamically recalculates the data bounds and redraws the plot within the new limits. When the C# GUI detects an axis change, it builds a JSON request to the F# plotting engine which returns the set of x and y values to plot. This design enforces a clear separation between the interface logic and the computational engine, allowing component development and testing to be done independently of each other.

3.8 Sprint 8: C Transpiler

3.8.1 BNF

The BNF required no update for the C transpiler to be implemented.

```
<Prog>      ::= <S> <Progopt>
<Progopt>   ::= ";" <S> <Progopt> | <empty>
<S>         ::= Id "=" <Comp> | <Comp>
<Comp>      ::= <E> | <E> "==" <E> | <E> "<" <E> | <E> ">" <E>
<E>         ::= <T> <Eopt>
<Eopt>      ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
<T>         ::= <P> <Topt>
<Topt>      ::= "%" <NR> <Topt> | "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<P>         ::= <F> <Popt>
<Popt>      ::= "^" <P> | <empty>
<F>         ::= <IfStmt> | <ForLoop> | <WhileLoop> | <NR> | <FCall>
<IfStmt>    ::= "if" "(" <Comp> ")" "then" "{" <Prog> "}" ("else" "{" <Prog> "}")?
```

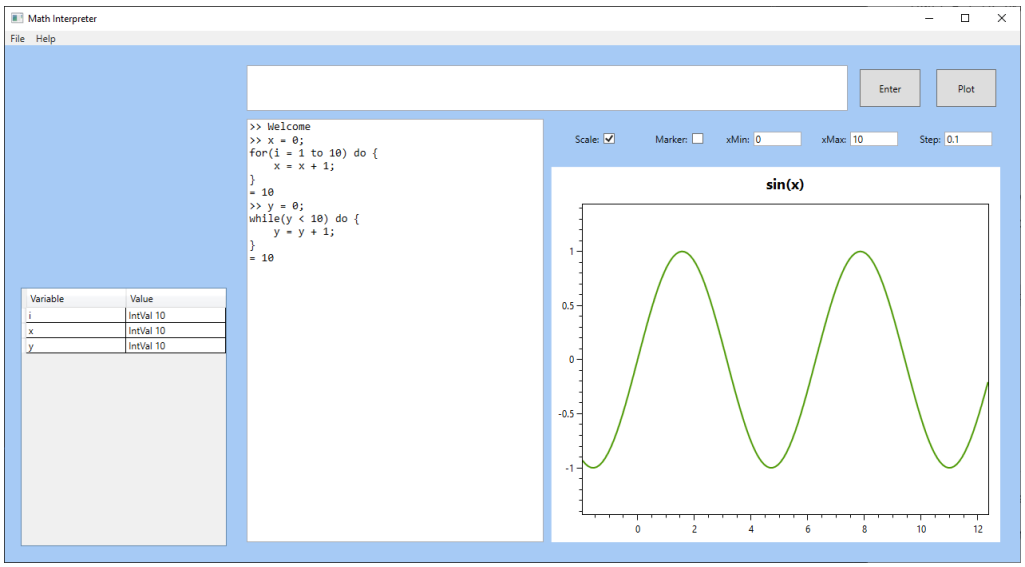

Figure 3.7: Sprint 7 GUI

```
<ForLoop>    ::= "for" "(" Id "=" <Comp> "to" <Comp> ")" "do" "{" <Prog> "}"
<WhileLoop>  ::= "while" "(" <Comp> ")" "do" "{" <Prog> "}"
<NR>         ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")" | Id
<FCall>      ::= Id "(" <Args> ")"
<Args>       ::= <Comp> <ArgList> | <empty>
<ArgList>    ::= "," <Comp> <ArgList> | <empty>
```

3.8.2 Updated GUI

The long awaited grid lines were finally added in this sprint to give better definition and readability to the user's plots. An additional read-only textbox was added to display the output of the C transpiler. The code that is produced by the C transpiler is also written to a `.c` file and saved to the `out/` folder in the project folder. Compilation to an executable was done an external C compiler by the user.

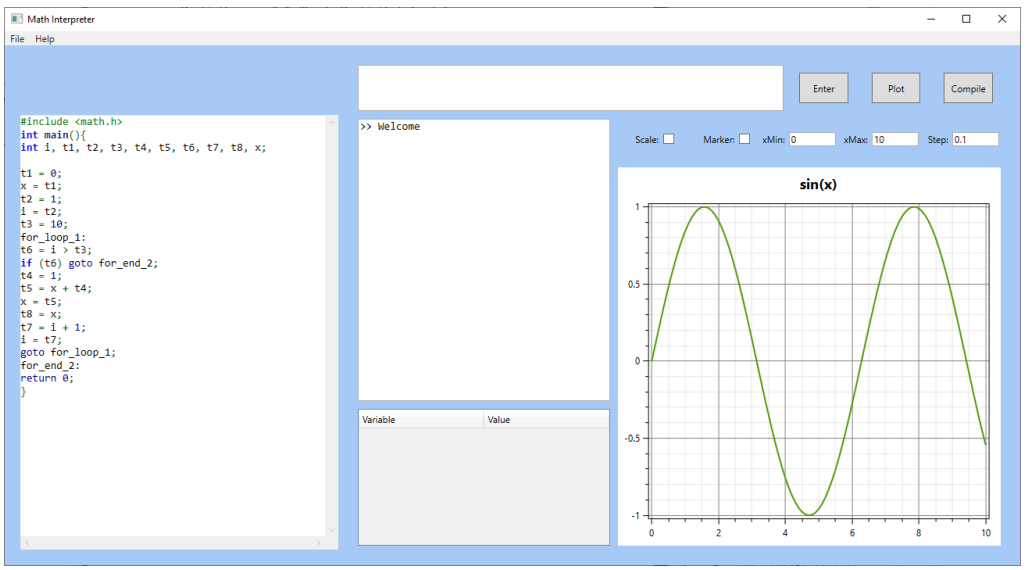

Figure 3.8: Sprint 8 GUI

3.8.3 C Transpiler

The C transpiler takes an Intermediate Representation (IR) produced by the frontend and translates it to another high level language, namely C. In compiler theory, the distinction between high and low level languages is based on the semantic abstractions provided by the language, rather than the traditional judgement of proximity to hardware. Sprint 8 focused on creating a system that adopts a clear front and back separation. The front end is responsible for parsing and semantic analysis, producing an Abstract Syntax Tree (AST) that is directly evaluated on by the interpreter. For compilation and transpilation, the AST is lowered into a Three Address Code (TAC) IR, which forms the interface to the back end. A simple C transpiler was implemented which features: variable assignment, function calls and control flow by the use of `goto` jumps. The code produced is not optimised; consequently, large programs would exhibit reduced performance compared to equivalent programs written directly in C.

3.9 Sprint 9: RISC-V Compiler

3.9.1 BNF

There were no changes for the BNF during this sprint.

```
<Prog>      ::= <S> <Progopt>
<Progopt>   ::= ";" <S> <Progopt> | <empty>
<S>         ::= Id "=" <Comp> | <Comp>
<Comp>      ::= <E> | <E> "==" <E> | <E> "<" <E> | <E> ">" <E>
<E>         ::= <T> <Eopt>
<Eopt>      ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
<T>         ::= <P> <Topt>
<Topt>      ::= "%" <NR> <Topt> | "*" <NR> <Topt> | "/" <NR> <Topt> | <empty>
<P>         ::= <F> <Popt>
<Popt>      ::= "^" <P> | <empty>
<F>         ::= <IfStmt> | <ForLoop> | <WhileLoop> | <NR> | <FCall>
<IfStmt>    ::= "if" "(" <Comp> ")" "then" "{" <Prog> "}" ("else" "{" <Prog> "}")?
<ForLoop>   ::= "for" "(" Id "=" <Comp> "to" <Comp> ")" "do" "{" <Prog> "}"
<WhileLoop> ::= "while" "(" <Comp> ")" "do" "{" <Prog> "}"
<NR>        ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")" | Id
<FCall>     ::= Id "(" <Args> ")"
<Args>      ::= <Comp> <ArgList> | <empty>
<ArgList>   ::= "," <Comp> <ArgList> | <empty>
```

3.9.2 Updated GUI

The GUI now allows the user to select the target language for transpilation/compilation. Furthermore, the software now has the ability to call the user's C compiler, in this case `gcc`. This simplifies the development process for the user by compiling their C code directly into an executable. The GUI manages the paths for file I/O and creates processes for the C compiler to run in. A run button was also created but has no function in this sprint.

3.9.3 C Transpiler

The C transpiler was optimised to reduce the number of temporary variables and intermediate operations generated during lowering. The previous approach translated a simple variable update into the following sequence:

```
Load X into T1
Compute T1 into T2
```

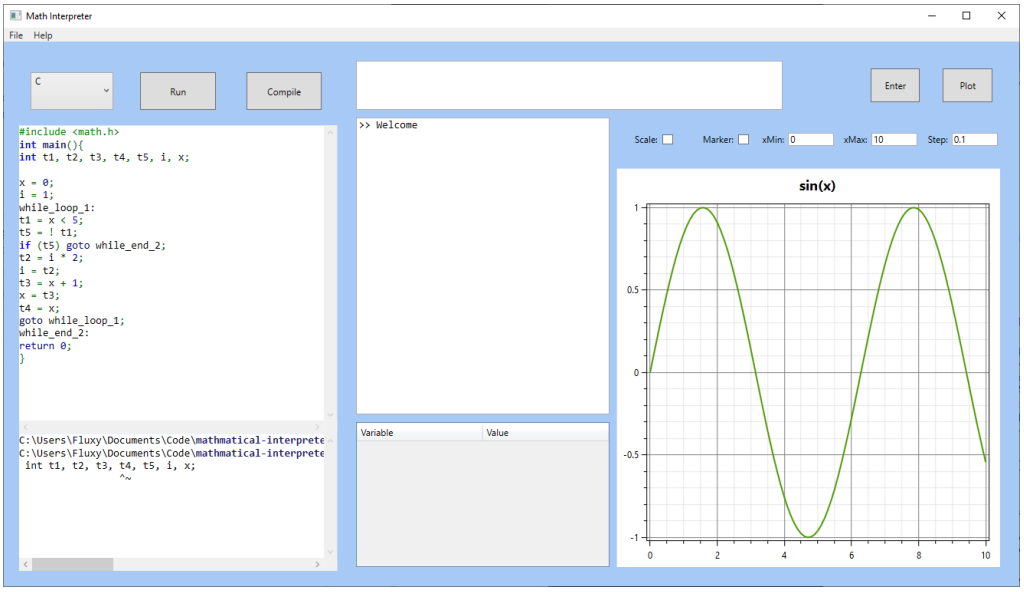

Figure 3.9: Sprint 9 GUI

```
Load T2 into X
```

While straightforward to generate, this approach required two temporary variables (`t1` and `t2`) and three intermediate operations to update a single variable. By eliminating the temporary values and performing the computation directly on `x`, the number of generated operations was reduced from three to one, representing a reduction of approximately 66%.

3.9.4 RISC-V Compiler

A naive RISC-V compiler was implemented during this sprint. The complexity of generating native assembly code is significantly higher than transpiling to another high level language. Registers are a finite resource and stack space must be pre-allocated prior to code generation. Effective stack allocation and stack management require complex analysis, including liveness analysis to determine variable lifetimes and avoid unnecessary memory spills. Many of these algorithms are beyond the scope of the project therefore, basic liveness analysis and a linear scan register allocation algorithm were implemented to create a working solution.

Chapter 4

Final deliverable

4.1 Final BNF

The final BNF implements user defined functions allowing users to define, store and compile their own functions. Functions take a list of arguments and a defines return value, see example below:

```
func add2(x, y) {
    sum = x + y;
    return sum;
}
```

The BNF allows for recursive function calls which are evaluated

```
func factorial(n) {
    if(n < 2) then {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}
```

```
<Prog>     ::= <S> <Progopt>
<Progopt>  ::= ";" <S> <Progopt> | <empty>
<S>        ::= "return" <Comp> | Id "=" <Comp> | <Comp>
<Comp>     ::= <E> | "==" <E> | "<" <E> | ">" <E>
<E>        ::= <T> <Eopt>
<Eopt>     ::= "+" <T> <Eopt> | "-" <T> <Eopt> | <empty>
<T>        ::= <P> <Topt>
<Topt>     ::= "%" <P> <Topt> | "*" <P> <Topt> | "/" <P> <Topt> | <empty>
<P>        ::= <F> <Popt>
<Popt>     ::= "^" <P> | <empty>
<F>         ::= <FuncDef> | <IfStmt> | <ForLoop> | <WhileLoop> | <NR> | <FCall>
<FuncDef>   ::= "func" Id "(" <Params> ")" "{" <Prog> "}"
<Params>    ::= Id <ParamList> | <empty>
<ParamList> ::= "," Id <ParamList> | <empty>
<IfStmt>    ::= "if" "(" <Comp> ")" "then" "{" <Prog> "}" ("else" "{" <Prog> "}")?
<ForLoop>   ::= "for" "(" Id "=" <Comp> "to" <Comp> ")" "do" "{" <Prog> "}"
<WhileLoop> ::= "while" "(" <Comp> ")" "do" "{" <Prog> "}"
<NR>       ::= "+" <NR> | "-" <NR> | "Num" <value> | "(" <E> ")" | Id
<FCall>    ::= Id "(" <Args> ")"
<Args>     ::= <Comp> <ArgList> | <empty>
<ArgList> ::= "," <Comp> <ArgList> | <empty>
```

4.2 Final GUI

The final GUI version, seen in Figure 4.1, overhauls the plot evaluation algorithm to avoid large jumps when plotting vertical asymptotes. The previous naive algorithm caught values close to infinity and returned a `NaN` value to OxyPlot that is skipped when rendering the plot. This naive behaviour caused lines to be draw across asymptotes whereas, the ideal behaviour is to capture the jump and generate multiple line series for a given function. The updated algorithm, see Algorithm 2, uses a relative jump to detect abnormal changes in values and return a list of line series. Ideally, the threshold would be linked to the axis scale however a fixed value of 0.5 works effectively.

The compiler view model now includes a `run` button to execute the most recently compiled binary. The output of the binary is displayed in the terminal output box. The run button spawns a new process that executes the compiled binary and returns the contents of `stdout` and `stderr`. Basic I/O was implemented so that the user could easier display results from their code using the print function. The print function automatically formats the interpolated string that `printf` requires. The execution of the compiled RISC-V code could not be implemented as the presence of a compatible system or emulator could not be guaranteed.

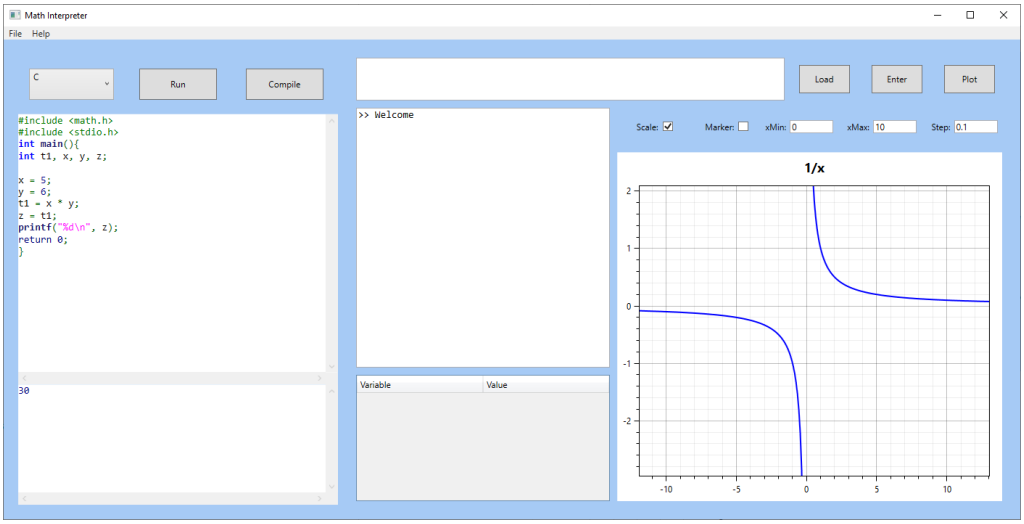

Figure 4.1: Final GUI

4.3 Final C Transpiler

4.3.1 Three Address Code

Three Address Code (TAC) is an intermediate representation that expresses all computation as a sequence of simple, linear instructions, each operating on at most three operands. Its primary purpose is to bridge the gap between high level, structured representations used in the front end of the compiler to the low level, machine specific instructions generated by the back end. By reducing expressions into a series of small, atomic operations, TAC provides a form that is easy to analyse during code generation.

Unlike the abstract syntax tree, which is hierarchical and deeply nested, TAC is flat and sequential. Attempting to generate low level assembly code on a tree data structure introduces unnecessary complexity and shifts the problem towards managing tree traversal rather than reasoning about the program's actual behaviour. The focus becomes handling recursion, evaluation order and intermediate storage during traversal, instead of concentrating on instruction selection, register usage and memory management. By first flattening the AST into a linear representation such as TAC, code generation is simplified into straightforward translation process, where each instruction maps naturally to a small

sequence of assembly operations and the emphasis on the substance of the program rather than the mechanics of navigating its structure. Each intermediate result is assigned to a temporary variable, the lifetime of every value becomes observable, which is essential for later stages such as liveness analysis and register allocation.

Optimisation is a large part of the compiler's function. By performing optimisations at the TAC level, much of the complexity is lifted away from the machine specific code generation stage. Instead of intertwining optimisation logic with instruction selection and register management, transformations can be applied directly applied to the TAC where program structure and data flow are explicit. This compiler focuses on TAC optimisations as machine level optimisations are complex and out of the scope of this project. The first naive implementation of the AST to TAC flatten transformation required all values to be loaded into a temporaries before an operation therefore, the expression `x` $= 5 + 3$ would be transformed into the TAC list:

```
t1 := 5
t2 := 3
t3 := t1 + t2
x := t3
```

This TAC list has two unnecessary load operations into temporaries `t1` and `t2`. The latest version of the compiler removes these unnecessary loads and computes the addition directly on the immediate values:

```
t1 := 5 + 3
x := t1
```

There are numerous TAC optimisations that could be implemented with the common goal of of reducing instruction count, minimising temporaries and simplifying data flow. These optimisations operate on a machine independent representation allowing optimisations to applied consistently across all target architectures. In contrast, machine level optimisations require detailed knowledge of hardware including instruction latency, pipeline behaviour and memory hierarchy. By performing most optimisations at the TAC level, the compiler can achieve significant efficiency improvements independent of the target system.

4.3.2 Type Inference and Promotion

The transpiler implements a dynamic typing system at the syntax level while performing compile-time type inference during TAC generation. This approach maintains the simplicity of a dynamically typed language for the user while generating statically typed C code with explicit type declarations.

Type inference is achieved through a `typeMap` data structure that tracks the inferred type of each variable and temporary during the lowering process. When a binary operation is encountered, the `inferBinaryType` function examines both operands and applies promotion rules: if either operand is a double, the result is double; otherwise, the result is an integer. This ensures that mixed-type expressions are handled correctly without requiring explicit type annotations in the source language.

The type promotion hierarchy follows the standard mathematical convention: integers promote to floating point values, and both promote to complex numbers when necessary. Complex number arithmetic presented a unique challenge as C does not provide native complex number support. The transpiler handles this by generating explicit arithmetic operations using the mathematical definitions. For example, complex multiplication $(a + bi)(c+di) = (ac-bd)+(ad+bc)i$ is expanded into separate real and imaginary component calculations in the generated C code.

4.3.3 User-Defined Functions

User-defined functions required careful consideration of both the interpreter and transpiler. Functions are stored in a `userFunctions` map during parsing, with each entry containing the parameter list and function body as an expression list. When the transpiler encounters a function definition, it must first compile the function body to determine its return type through type inference. This creates a dependency: the function's type signature cannot be generated until after the body has been analyzed.

To resolve this, the transpiler performs a two-pass approach. First, it collects all function definitions from the program. Then, for each function, it temporarily compiles the body in isolation, tracking the type of the return value through the `typeMap`. Once the return type is known, the transpiler generates a complete C function declaration with the inferred return type and parameter types. This allows recursive functions to reference themselves correctly, as the function signature is known before code generation begins.

The generated C functions follow standard calling conventions, with parameters passed by value and the return statement mapping directly to C's return semantics. Function bodies are translated into TAC and then converted to C code using the same pipeline as the main program, ensuring consistency between user-defined and built-in functionality.

4.4 Final RISC-V Compiler

4.4.1 Liveness Analysis

Designing a compiler raises complications that a transpiler can defer to the user's toolchain. Since resources are finite immediate values, temporaries and variables must be analysed during before the code generation stage to determine where they will reside during execution. Values may exist at three locations: hardware registers, the stack and the heap. This compiler restricts its scope to hardware registers and the stack.

To make effective use of the limited number of hardware register, the compiler performs liveness analysis on the Three Address Code (TAC) representation. Liveness analysis determines for any given point of the program which variables or temporaries may be used in the future. A value is determined live if it is read again and dead if it is not referenced further. This information is needed to safely assign values to hardware registers without overwriting live values. An example program is shown below:

```
t1 = 5        ; a = 5
t2 = 10       ; b = 10
t3 = t1 + t2  ; c = a + b
t4 = t2 + 2   ; d = b + 2
t5 = t1 + 2   ; e = a + 2
```

Given a target machine with three hardware registers, a naive compiler may produce:

```
R1 = 5          ; t1 = 5
R2 = 10         ; t2 = 10
R3 = R1 + R2    ; t3 = t1 + t2
R1 = R2 + 2     ; t4 = t2 + 2
R2 = R1 + 2     ; t5 = t1 + 2
```

The generated code shows the primary issue of compilation without liveness analysis. `t1` is still live after it computed into `R3`, therefore, `t4` being assigned to `R1` invalidates the value of `t1`. Liveness analysis would identify this dependency and produce:

```
R1 = 5          ; t1 = 5
R2 = 10         ; t2 = 10
R3 = R1 + R2    ; t3 = t1 + t2
R2 = R2 + 2     ; t4 = t2 + 2
R1 = R1 + 2     ; t5 = t1 + 2
```

This code ensures that data is valid at the point of any given temporary being live. Using the output of liveness analysis, the compiler assigns frequently used values to registers while, longer lived and less frequently used values are spilled into the stack. This reduces the use of the slower stack access and maximises the use of fast hardware registers in turn improving code performance.

Algorithm 1 provides a pseudocode representation of liveness analysis. The algorithm works on a basic block, a sequence of instructions that has a single entrance and exit with no branches, and maintains:

- `USE[B]`: variables used in a block before being defined
- `DEF[B]`: variables assigned a value in the block
- `OUT[B]`: variables live at the exit of the block
- `IN[B]`: variables live at the entry of the block

The algorithm iteratively updates `IN` and `OUT` sets until they stabilise.

- `OUT` is computed as the union of the `IN` sets of its successors block
- `IN` is assigned as the union of `USE[B]` with the variables in `OUT[B]` that are not redefined in the block `(OUT[B] - DEF[B])`

This process propagates liveness information backwards where a variable is live at the entry of a block if it is used in the block or it is live at the exit and is not redefined. This process continues until the sets converge.

Optimisations at the hardware level is something that a programmer cannot control meaningly therefore, the compiler designer is responsible for

4.4.2 Linear Scan Register Allocation

Linear scan register allocations assigns hardware registers to temporaries and variables based on their live ranges. Using the output of liveness analysis, linear scan selects from a set of available registers which are then recalled during code generation. Due to the limited set of hardware registers available, there are cases where the set of live values is greater than the set of available registers. This case must be captured and handled by assigning the value a stack slot.

Algorithm 3 is a pseudocode representation of this process. This algorithm processes temporaries in order of appearance, assigning registers greedily and spilling to the stack when register pressure exceeds hardware limits. The linear scan algorithm is fast, simple and has low overhead however, often overspills to stack allocation unlike optimal graph colouring algorithms. Each temporary is represented by a live interval `[start, end]` describing the program region in which it must be preserved. The `Active` set represents the current register pressure of the program and if the size of this set exceeds the number of available registers, spilling is unavoidable, and the allocator must choose a value to evict. The values with the furtherest interval end points are usually chosen as it minimises the likelihood of future reloads. When a value is spilled, a slot is reserved on the stack frame and all future uses of that temporary are rewritten as loads from memory, while definitions are emitted as stores. This introduces greater memory traffic on the slower stack memory however, guarantees correctness when register pressure is high. The final stack frame size must be determined prior to execution therefore, this process must be done during the compilation stage and not at runtime. RISC-V contains seven caller-saved temporary (`t0-t6`) and, optionally, twelve saved registers (`s0-s11`). This compiler restricts allocation to caller-saved temporary registers to reduce complexity and avoid the cost of additional prologue and epilogue instructions. The more registers available to the compiler, the less frequently a spillage to the stack will occur and in turn an increase in performance.

While graph colouring allocators can produce more optimal register assignments, their implementation is significantly higher and therefore, out of the scope of this project. A linear scan approach offers a practical trade off between performance and simplicity, making it well suited for the simple approach to a compiler this project aims to implement. Furthermore, the small overhead and linear time complexity allow allocation to scale efficiently with program size while still remaining easy to understand and debug. This project argues that accurate liveness analysis has a greater impact on overall performance than the use of a more complex optimal allocator.

4.5 Code architecture

4.5.1 GUI

Fig. shows a UML class diagram (class, sequence and state diagrams are the most frequently used UML diagrams). Illustrating your code architecture - that should be of the MVC family and, considering it is developed in C# with WPF more specifically the MVVM pattern - is very important.

The GUI follows the Model-View-ViewModel (MVVM) architecture as illustrated by Figure C.1. Each view relates to a graphic interface window (main, help and about) which has a logical link to it's respective View Model, as seen in Figure C.2. The `MainViewModel` interacts with a `CompilerViewModel`, `REPLViewModel` and `PlotViewModel` that logically separates the functionality of the GUI. Each ViewModel uses one or many Services that provide functions to the ViewModel. For example, the `REPLViewModel` operates the Read-Evaluate-Print-Loop section of the GUI by using the `ComputeService`, `SymbolTableService` and `ConsoleService` to manage interpreter I/O. Each Service has an Interface to ensure sufficient functionality of the Service implementation. Figure C.3 displays the complete MVVM system that was implemented and the interactions between each ViewModel and Services. This design encourages logical separation of concerns and tries to isolate functionality to a single Service that is standardised across each ViewModel, reducing duplicate code and scope creep.

This architecture improves testability by allowing the ViewModels and Services to be validated independently. Services do not store information, excluding some string constants, therefore the service is able to be treated as a device under test (DUT). Services can be substituted during unit testing, enabling isolation of the compiler, plotting and REPL logic from the GUI. This allows core functionality to be tested without the requirement of the GUI to be active. The use of Services also improves the modularity of the software, extending the scope to allow new services and functionality without having to modify the surrounding ViewModels. Similarly, alternative versions of services can be introduced since services are exclusively accessed through their interface therefore, guaranteeing legacy functionality if the interface remains constant.

4.5.2 F#

Unlike the C# GUI, the F# language follows the functional programming language paradigm. Figure C.4 show the data flow diagram for the language's front and back end. Instead of interactions between classes and their methods, the F# code is structured as a pipeline of transformations, where each stage maps one representation of the program to another. Each stage is represented as a pure function, allowing each stage to be tested individually. Each stage operates on a defined data structure making the program representation explicit and immutable. This structured approach creates clear interactions between two stages of the pipeline and prevents unintended side effects. The code is split into two high level modules: the front end and the back end. Each module has individual submodules that take a structure of the previous data type as input and outputs a structure of its own data type. The guarantee of data type between stages allows individual components to be replaced or extended without impacting the rest of the

system. This is especially prevalent during optimisation of the system where optimisation passes of the system can be introduced as independent transformations on an intermediate representation. Since these passes operate on a well defined IR, multiple passes can be composes, reordered or removed without the surrounding stages being affected. This allows for incremental performance increases while preserving the overall compiler structure, and ensures optimisation remains a modular concern rather than being intertwined in parsing and code generation.

Discriminated unions are extensively used to represent the program during each stage. A discriminated union represents a value that could be one of a group of defined types. For example, a `NumericValue` data type could represent an integer, float or complex value.

```
type NumericValue =
    | IntVal of int
    | FloatVal of float
    | ComplexVal of float * float
```

It also allows the compiler to enforce semantic and syntactic rules directly into the data types rather than relying on runtime checks. Combined with the common practice of pattern matching, the compiler is forced to validate all possible cases at compile time, reducing the risk of unhandled scenarios and improving maintainability as features are introduced. Each stage owns a specific discriminated union guaranteeing the structure and type of its input and output. This approach simplifies debugging and testing as each stage has a well defined and strongly typed structure that can be inspected directly, rather than being inferred from loosely structured data.

Pattern matching is used throughout the program to drive control flow over each stage. This replaces large nested conditional chains, that grow exponentially with feature introduction, improving readability and maintainability. Furthermore, traditional iteration tooling is replaced with function composition and piping to express transformations as a linear flow of data. Instead of relying on imperative control structures such as for loops and mutable state, high order functions such as `List.map`, `List.fold` and `List.filter` are used to describe iterative computation. This focuses the design on what is being computer rather than how it is being executed, improving readability and reducing implementation complexity. By expressing these stages as composable functions, the entire compilation process can be written as a single data flowing using the pipe operator `|>`. The functional programming style closely mirrors the structure of a compiler, where each stage transforms one representation into another, making the implementation feel natural and manageable rather than a constant battle with low level errors and edge cases.

The F# code is callable by the C# GUI for each of the three data flows and returns a standardised JSON serialised string which header describes the nature of the data. Since the core of the compiler is implemented in a self contained project, the F# code can be tested in isolation to the GUI removing the requirement of a graphical interface for unit testing. Creating a clear logical separation of concerns removes the development dependency of each other allowing both to progress at their own rate.

4.5.3 Testing

Due to the modular architecture of the software, each component of the software was able to be tested in isolation. Therefore, thanks to the separation of concerns, we can test syntactical errors once across all three mediums as they share a common front end. Table B.1 shows the tests completed for the language syntax. To develop tests, each line of the BNF was studied and scrutinised using expressions that were made to break. Each error must be caught at the lexer or parser level and not at execution.

Table B.2 shows the testing table that was used while testing the interpreter. Unlike Table B.1, the errors must be caught at the evaluation stage. Automatic unit tests were ran using the .NET `TestCase`. The automation of testing made development more efficient as manual testing is slow and unreliable due to the human input required.

Table B.5 shows the set of functions that were plotted to test the plotting engine. Each plot was manually compared against Desmos [Desmos Studio PBC, 2023], a widely used graphical calculator, to verify correctness and identify any inconsistencies or visual bugs. Plot testing proved to be the most laborious part of the testing process due to the lack of automation support: OxyPlot does not provide built-in functionality to programmatically compare rendered graphs, meaning that every function and its corresponding output had to be visually inspected. This manual approach was particularly important for detecting subtle issues such as incorrect evaluation, asymptotes, or rendering faults. Despite being time consuming, these tests were essential to ensure the reliability of the plotting engine, as graphical correctness cannot be captured through traditional unit tests alone.

Chapter 5

Discussion, conclusion and future work

5.1 MoSCoW Analysis Reflection

The final version of this project's software matched the functional requirements outlined in Table 1.1. All `Must` requirements were successfully implemented, with both the interpreter and C transpiler achieving a high standard of functionality. We also completed several `Should` requirements, including more advanced language features such as control flow and support for complex numbers. In hindsight, incorporating additional mathematical features would have strengthened the language. Given the mathematical nature of the language, including specialised features not commonly found in other programming languages would have distinguished it further. While the initial scope prioritised demonstrating compiler theory, which allowed some flexibility in the mathematical aspects, future development would have allowed these features to be implemented, enhancing both the functionality and uniqueness of the language. We are proud to have been able to complete the `Could` requirements, most notably, the RISC-V compiler. While the compiler does not have comparable functionality to the interpreter and compiler, the complexity of a compiler was felt during the implementation. The liveness analysis and register allocation features are significant milestones to implementing a fully functional compiler and provide a solid base for further development to introduce features such as floating point values and user defined functions.

5.2 Development Reflection

The functional programming paradigm was a new concept for both team members, so our understanding of functional development evolved throughout the project. Over time, this growth led to many features being rewritten in a purely functional style. We are proud to have eliminated all non-functional code, such as traditional for loops, resulting in a consistent and idiomatic functional implementation across the project.

The MVVM approach simplified the implementation of the GUI and allows us to focus most of the development time on the F# core while still producing a responsive and clear GUI. Separating the GUI logic from the core computations meant that changes to the backend or frontend could be made independently without the risk of regressions and making debugging more straightforward.

One of the key challenged faced during the project was integrating multiple stages of the compiler into a cohesive pipeline. The dependencies between stages was not always immediately obvious, requiring careful design of data structures. However, the use of immutable data and well defined intermediate representations made it easier to reason about program state and to test components individually. Compiler theory was supplied by the classical techniques outlined in [Aho et al., 2006], which provides solid foundations on the complete compiler pipeline from lexing to code optimisation. While the practices

outline may be dated, it provided a level of complexity that suited the scope of this project being our first forte into compiler design.

Bibliography

[Aho et al., 2006] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison-Wesley, 2nd edition.

[Andreas Rumpf, 2026] Andreas Rumpf (2026). Nim website. https://nim-lang.org/ [Accessed: 06/01/2026].

[AvaloniaUI OÜ, 2026] AvaloniaUI OÜ (2026). Avalonia ui. https://avaloniaui.net/ [Accessed: 06/01/2026].

[Desmos Studio PBC, 2023] Desmos Studio PBC (2023). Desmos website. https://desmos.com [Accessed: 30/11/2023].

[Free Software Foundation, 2024] Free Software Foundation (2024). *GNU Bash Reference Manual*. Accessed: 2025-03-08.

[JuliaHub, Inc., 2026] JuliaHub, Inc. (2026). Julia programming language. https://julialang.org [Accessed: 6/1/2026].

[MathWorks ®, 2026a] MathWorks ® (2026a). Matlab system requirements. https://uk.mathworks.com/support/requirements/matlab-system-requirements.html [Accessed: 06/01/2026].

[MathWorks ®, 2026b] MathWorks ® (2026b). Matlab website. https://uk.mathworks.com/products/matlab.html [Accessed: 06/01/2026].

[Microsoft Learn, F#, 2026] Microsoft Learn, F# (2026). *Microsoft Visual Basic Documentation*. Microsoft.

[Microsoft Learn, Visual Basic, 2026] Microsoft Learn, Visual Basic (2026). *Microsoft Visual Basic Documentation*. Microsoft.

[Microsoft Learn, WPF, 2026] Microsoft Learn, WPF (2026). *Windows Presentation Foundation documentation*. Microsoft.

[OxyPlot, 2026] OxyPlot (2026). Oxyplot website. https://oxyplot.github.io/ [Accessed: 6/1/2026].

[Python Software Foundation, 2024] Python Software Foundation (2024). *The Python Tutorial*. Accessed: 2025-03-08.

Appendix A

Contributions

The contribution to the project was agreed to be an equal 50% per member. The project was split into: GUI, interpreter, transpiler and compiler to make sure we were not interfering with each other's work. Both members started work on the interpreter and as the project grew we focused our work on particular areas of the project. Ali focuses work on the transpiler and the mathematical features of the interpreter and I focused work on the GUI and compiler. Even though we were working on separate areas of the project, we made sure to meet during lab sessions and continue weekly catch-up calls over the break to check progress and attend to any blockers.

Appendix B

Testing

B.1 Language testing

Table B.1: Syntax tests

Expression	ResE	ResA	Pass/Fail	Action/comment
$(5 * (2 - 3)))$	Error	Error	Pass	Trailing bracket
$(5 * (2 - 3)$	Error	Error	Pass	Missing trailing bracket
$()$	Error	Error	Pass	Empty parentheses
$5 === 3$	Error	Error	Pass	Invalid operator
$return == 5$	Error	Error	Pass	Invalid return syntax
$return = 5$	Error	Error	Pass	Reserved word assignment
$x = 5$	x= 5	x= 5	Pass	Valid assignment
$3 = 5$	Error	Error	Pass	Left-hand side must be Id
$3 + 5$	8	8	Pass	Valid addition
$3 + +5$	Error	Error	Pass	Invalid operator sequence
$5-$	Error	Error	Pass	Missing right operand
^2	Error	Error	Pass	Missing left operand
2^^2	Error	Error	Pass	Double exponent operator
2^	Error	Error	Pass	Missing right operand
$((5))$	5	5	Pass	Nested parentheses
$(5+)$	Error	Error	Pass	Missing operand in expression
$f()$	Call	Call	Pass	Empty argument list
$f(, 5)$	Error	Error	Pass	Leading comma in arguments
$f(5,)$	Error	Error	Pass	Trailing comma in arguments
$func\ f()\ \{return\ 5\}$	Func	Func	Pass	Valid function definition
$func()\ \{return\ 5\}$	Error	Error	Pass	Missing function identifier
$if(5)\ then\ \{return\ 1\}$	1	1	Pass	Valid if statement
$if\ 5\ then\ \{return\ 1\}$	Error	Error	Pass	Missing parentheses in if
$for(i = 0\ to\ 10)\ do\ \{x = 1\}$	Ok	Ok	Pass	Valid for loop
$for(i =\ to\ 10)\ do\ \{x = 1\}$	Error	Error	Pass	Missing initial value
$while(x)\ do\ \{x = x - 1\}$	Ok	Ok	Pass	Valid while loop
$while(x)\ \{x = x - 1\}$	Error	Error	Pass	Missing `do` keyword

B.2 Arithmetic expression testing

B.2.1 Interpreter

Table B.2: Arithmetic expression tests

Expression	ResE	ResA	Pass/Fail	Action/comment
$5*3+(2*3-2)/2+6$	23	23	Pass	
$9-3-2$	4	4	Pass	left assoc.
$10/3$	3	3	Pass	int division
$10/3.0$	3.333	3.333	Pass	float division
$10\%3$	1	1	Pass	
$10--2$	12	12	Pass	unary minus
$-2+10$	8	8	Pass	
$3*5\hat{}(-1+3)-2\hat{}2*-3$	87	87	Pass	power test
$-3\hat{}2$	-9(*) or 9	9	Pass	precedence
$-7\%3$	2(*) or -1	-1	Pass	precedence (*)Python
$2*3^2$	18	18	Pass	precedence pow ¿ mult
$3*5\hat{}(-1+3)-2\hat{}-2*-3$	75.750 or 75	75.750	Pass	
$3*5\hat{}(-1+3)-2.0\hat{}-2*-3$	75.750	75.750	Pass	
$(((3*2--2)))$	8	8	Pass	
$(((3*2--2))$	Error	Error	Pass	syntax error
$-((3*5-2*3))$	-9	-9	Pass	minus expression
$x=3;(2*x)-x\hat{}2*5$	-39	-39	Pass	var assign
$x=3;(2*x)-x\hat{}2*5/2$	-16	-16	Pass	
$x=3;(2*x)-x\hat{}2*(5/2)$	-12	-12	Pass	
$x=3;(2*x)-x\hat{}2*5/2.0$	-16.5	-16.5	Pass	
$x=3;(2*x)-x\hat{}2*5\%2$	5	5	Pass	
$x=3;(2*x)-x\hat{}2*(5\%2)$	-3	-3	Pass	
complex(2,3) $*-2$	$-4.0-6.0i$	$-4.0-6.0i$	Pass	
complex(1,-1) + complex(2,3)	$3.0+2.0i$	$3.0+2.0i$	Pass	addition
complex(2,3) − complex(1,1)	$1.0+2.0i$	$1.0+2.0i$	Pass	subtraction
$a=$ complex(3,4); magnitude(a)	5.0	5.0	Pass	magnitude calculation
$a=$ complex(3,4); conjugate(a)	$3.0-4.0i$	$3.0-4.0i$	Pass	conjugate calculation
$((x=2);(y=3);(x+y)\hat{}2)$	25	25	Pass	multiple variable assignment and power
$((x=2);(y=3);x\hat{}y+y\hat{}x)$	17	17	Pass	variable exponentiation

B.2.2 C Transpiler

Table B.3: Arithmetic expression tests - C Transpiler

Expression	ResE	ResA	Pass/Fail	Action/comment
$5 * 3 + (2 * 3 - 2)/2 + 6$	23	23	Pass	
$9 - 3 - 2$	4	4	Pass	left assoc.
$10/3$	3	3	Pass	int division
$10/3.0$	3.333	3.333	Pass	float division
$10\%3$	1	1	Pass	
$10 - -2$	12	12	Pass	unary minus
$-2 + 10$	8	8	Pass	
$3 * 5\hat{}(-1 + 3) - 2\hat{}2 * -3$	87	87	Pass	power test
$-3\hat{}2$	-9(*) or 9	9	Pass	precedence
$-7\%3$	2(*) or -1	-1	Pass	precedence (*)Python
$2 * 3^2$	18	18	Pass	precedence pow mult
$3 * 5\hat{}(-1 + 3) - 2\hat{} - 2 * -3$	75.750 or 75	75.750	Pass	
$3 * 5\hat{}(-1 + 3) - 2.0\hat{} - 2 * -3$	75.750	75.750	Pass	
$(((3 * 2 - -2)))$	8	8	Pass	
$(((3 * 2 - -2))$	Error	Error	Pass	syntax error
$-((3 * 5 - 2 * 3))$	-9	-9	Pass	minus expression
$x = 3; (2 * x) - x\hat{}2 * 5$	-39	-39	Pass	var assign
$x = 3; (2 * x) - x\hat{}2 * 5/2$	-16	-16	Pass	
$x = 3; (2 * x) - x\hat{}2 * (5/2)$	-12	-12	Pass	
$x = 3; (2 * x) - x\hat{}2 * 5/2.0$	-16.5	-16.5	Pass	
$x = 3; (2 * x) - x\hat{}2 * 5\%2$	5	5	Pass	
$x = 3; (2 * x) - x\hat{}2 * (5\%2)$	-3	-3	Pass	

B.2.3 RISC-V Compiler

Table B.4: Arithmetic expression tests - RISC-V Compiler

Expression	ResE	ResA	Pass/Fail	Action/comment
$5 * 3 + (2 * 3 - 2)/2 + 6$	23	17	Fail	
$9 - 3 - 2$	4	4	Pass	left assoc.
$10/3$	3	3	Pass	int division
$10/3.0$	3.333	Error	Fail	Compiler doesn't support loading floating point
$10\%3$	1	1	Pass	
$(((3 * 2 - -2)))$	8	Error	Fail	Compiler doesn't support unary minus

```
// Control flow test
x=2;
y=3;
z=0;
if(x < y) then {
    z = 5
} else {
    z=10
}
// Expected result = 5
// Actual result = 5
```

B.2.4 Code Snippet Tests

Larger test programs were run to test specific features of the code on both the interpreter and C transpiler. The output shows the final value output of the values being tested.

For loop control flow

```
x = 0
for(i = 1 to 10) do {
    x = x + 1;
}

// Expected output: x = 10 ;i = 10
// Actual output: x = 10 ;i = 10
```

While loop control flow

```
i = 0
while(i < 10) do {
    i = i + 1;
}

// Expected output: i = 10
// Actual output: i = 10
```

Infinite while loop control flow

```
i = 0
while(1) do {
    i = i + 1;
}

// Expected output: Error
// Actual output: Crash
```

User defined function

```
func add2(a,b) {
    return a + b;
}

y = add(2,5)
// Expected output = 10
// Actual output = 10
```

Recursive function call test

```
func factorial(n) {
    if(n < 2) then {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

y = factorial(5)
// Expected output = 120
```

```
// Actual output = 120
```

Nested loop control flow

```
sum = 0;
for(i = 1 to 3) do {
    for(j = 1 to 3) do {
        sum = sum + 1;
    }
}
// Expected output: sum = 9
// Actual output: sum = 9
```

Function with local variables and control flow

```
func max(a, b) {
    result = 0;
    if(a > b) then {
        result = a;
    } else {
        result = b;
    }
    return result;
}
x = max(10, 25);
// Expected output: x = 25
// Actual output: x = 25
```

Function composition

```
func square(x) {
    return x * x;
}
func add(a, b) {
    return a + b;
}
func sumOfSquares(a, b) {
    return add(square(a), square(b));
}
result = sumOfSquares(3, 4);
// Expected output: result = 25
// Actual output: result = 25
```

Loop with function call

```
func square(x) {
    return x * x;
}
sum = 0;
for(i = 1 to 5) do {
    sum = sum + square(i);
}
// Expected output: sum = 55 (1+4+9+16+25)
// Actual output: sum = 55
```

Power function using recursion

```
func power(base, exp) {
    if(exp == 0) then {
        return 1;
    } else {
        return base * power(base, exp - 1);
    }
}
result = power(2, 10);
// Expected output: result = 1024
// Actual output: result = 1024
```

Greatest common divisor (Euclidean algorithm)

```
func gcd(a, b) {
    if(b == 0) then {
        return a;
    } else {
        return gcd(b, a % b);
    }
}
result = gcd(48, 18);
// Expected output: result = 6
// Actual output: result = 6
```

B.3 GUI testing

The GUI tests were completed to ensure the application interface is functional and responsive. This included verifying button inputs and input validation. Testing consisted of manual exploratory testing and functional testing while also testing the evaluator. Fully automated GUI testing could be implemented in the future however, the GUI is simple and implementation of automated testing would be unnecessary for our use case. The GUI testing confirmed that all interactive elements responds correctly to the user actions, results are accurately displayed and invalid inputs are handled.

B.4 Plot testing

Table B.5: Plot evaluation tests

Expression	Pass/Fail	Action/comment
$y = 2x$	Pass	Linear
$y = x^2$	Pass	Polynomial
$y = x^2 + 3x + 5$	Pass	Quadratic
$y = x^3 + x^2 + 3x + 5$	Pass	Cubic
$y = sin(x)$	Pass	Trigonometric function
$y = cos(x)$	Pass	Trigonometric function
$y = tan(x)$	Pass	Trigonometric function with vertical asymptote. Tune rolling threshold for smooth plot segmenting
$y = sin(1/x)$	Pass (*)	High frequency oscillations at $x = 0$. *Successful using a step small enough to detect oscillations
$y = sinh(x)$	Fail	Exponential growth function. Plotting crashes as it tends to infinity during zooming out
$y = cosh(x)$	Pass (*)	Exponential growth function. *Intermittent line drawing yet data points are valid
$y = 1/x$	Pass	Vertical asymptote
$x = 5$; $y = 2x$;	Pass	Symbol table environment conflict
$x * sin(x)$	Pass (*)	Oscillations increase with x. *Incorrectly detected jumps when step > 0.1
$\|x\|$	Pass	Absolute function
if$(x > 0)$then$\{x\}$	Pass	Conditional plotting. All values of $x > 0$ else plot 0
if$(x > 0)$then$\{sin(x)\}$	Pass	Conditional plotting with trigonometric functions

Appendix C

Diagrams

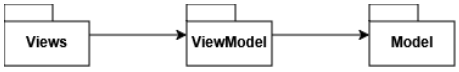

Figure C.1: MVVM Paradigm

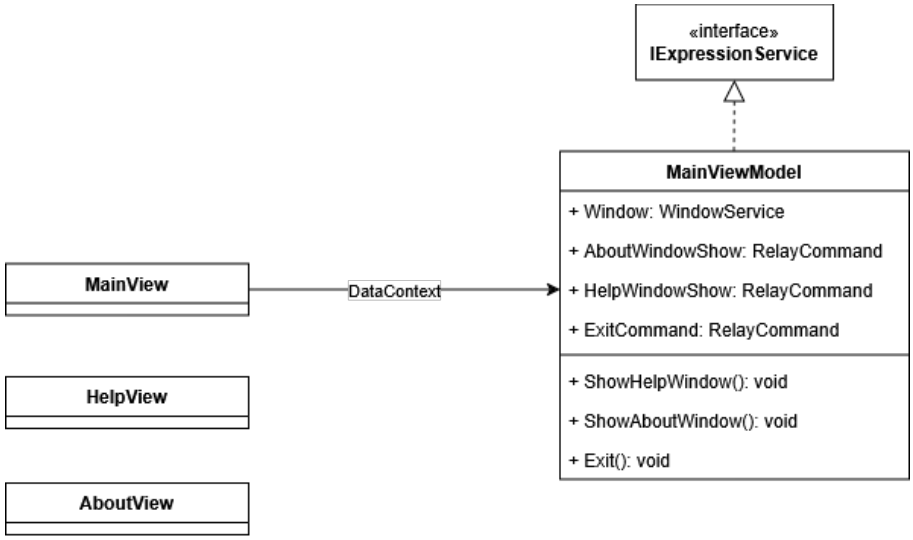

Figure C.2: View to ViewModel Relationship

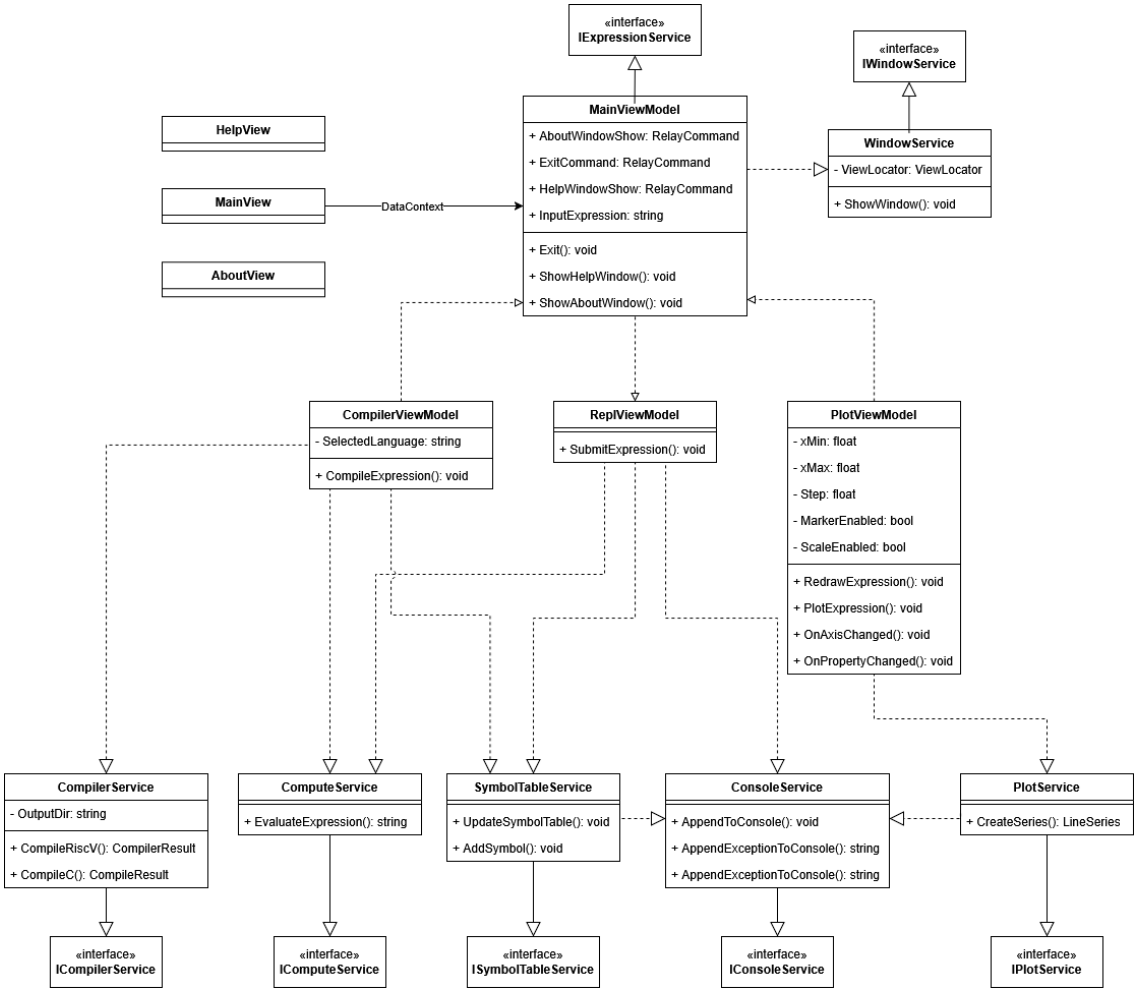

Figure C.3: GUI MVVM UML Diagram

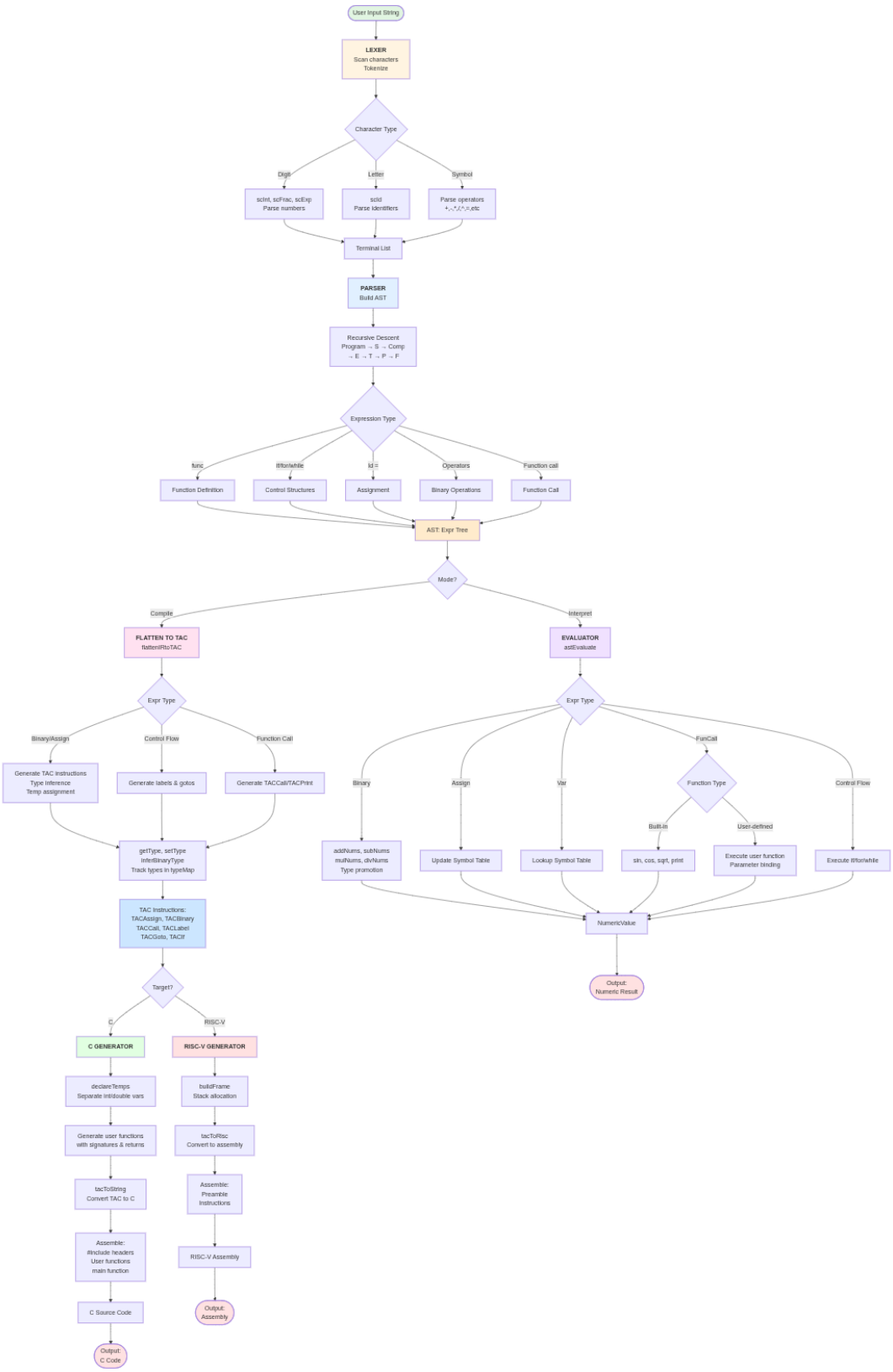

Figure C.4: F# Data Flow Diagram

Appendix D

Algorithms

Algorithm 1 Liveness Analysis for Three Address Code (TAC)

1: Initialise `IN` and `OUT` sets for each block as empty
2: Compute `USE` and `DEF` sets for each block
3: **repeat**
4: **for all** basic blocks B in the control flow graph **do**
5: `OUT[B]` $:= \cup_{S \in \text{succ}(B)}$ `IN[S]`
6: `IN[B]` $:=$ `USE[B]` $\cup$ (`OUT[B]` $-$ `DEF[B]`)
7: **end for**
8: **until** `IN` and `OUT` sets do not change
9: Return `IN` and `OUT` sets for all blocks

Algorithm 2 Relative Jump Segmentation for Plotting

Initialise list of segments $S \leftarrow [\,]$
Initialise current segment $C \leftarrow [\,]$
for all points (x_i, y_i) in order **do**
  if y_i is NaN or y_i is $\pm\infty$ **then**
    if $C \neq [\,]$ **then**
      Append C to S
      $C \leftarrow [\,]$
    end if
    continue
  end if
  if $C = [\,]$ **then**
    $C \leftarrow [(x_i, y_i)]$
  else
    Let (x_{i-1}, y_{i-1}) be the last point in C
    `relativeJump` $\leftarrow \dfrac{|y_i - y_{i-1}|}{\max(|y_{i-1}|, 1)}$
    if `relativeJump` > 0.5 **then**
      Append C to S
      $C \leftarrow [(x_i, y_i)]$
    else
      Append (x_i, y_i) to C
    end if
  end if
end for
if $C \neq [\,]$ **then**
  Append C to S
end if
Return S

Algorithm 3 Linear Scan Register Allocation

```
1: Compute live intervals for all temporaries using liveness analysis
2: Sort all intervals by increasing start point
3: Active := empty list
4: FreeRegs := set of all available hardware registers
5: for all intervals v in sorted order do
6:    Remove expired intervals from Active:
7:    for all a ∈ Active do
8:       if a.end < v.start then
9:          Remove a from Active
10:         Add a.reg to FreeRegs
11:      end if
12:   end for
13:   if FreeRegs is not empty then
14:      Assign a register r from FreeRegs to v
15:      v.reg := r
16:      Insert v into Active, sorted by end point
17:   else
18:      Choose interval s ∈ Active with the largest end point
19:      if s.end > v.end then
20:         Spill s to stack
21:         v.reg := s.reg
22:         Remove s from Active
23:         Insert v into Active
24:      else
25:         Spill v to stack
26:      end if
27:   end if
28: end for
29: Return register and stack assignments for all intervals
```

Algorithm 4 Type Inference for Binary Operations

```
1: Input: Left operand l, right operand r, typeMap
2: Output: Inferred result type
3:
4: function inferBinaryType(l, r)
5:     t_l ← getType(l)
6:     t_r ← getType(r)
7:     if t_l = "complex" or t_r = "complex" then
8:         return "complex"
9:     else if t_l = "double" or t_r = "double" then
10:        return "double"
11:    else
12:        return "int"
13:    end if
14: end function
15:
16: function getType(operand)
17:    match operand with
18:        case OpImmInt: return "int"
19:        case OpImmFloat: return "double"
20:        case OpVar(name) or OpTemp(t):
21:            if name or t in typeMap then
22:                return typeMap[name] or typeMap[t]
23:            else
24:                return "int" (default)
25:            end if
26:    end match
27: end function
```

Algorithm 5 AST to Three Address Code Flattening

```
1: Input: Expression expr from AST
2: Output: List of TAC instructions, final operand
3:
4: function flattenIRtoTAC(expr)
5:   match expr with
6:     case Int(value):
7:       if value is integer then
8:         return ([], OpImmInt(value))
9:       else
10:        return ([], OpImmFloat(value))
11:      end if
12:
13:    case Var(name):
14:      return ([], OpVar(name))
15:
16:    case Binary(left, op, right):
17:      (code_L, op_L) ← flattenIRtoTAC(left)
18:      (code_R, op_R) ← flattenIRtoTAC(right)
19:      temp ← assignTemp()
20:      resultType ← inferBinaryType(op_L, op_R)
21:      setType(OpTemp(temp), resultType)
22:      instr ← TACBinary(OpTemp(temp), op_L, op, op_R)
23:      return (code_L || code_R || [instr], OpTemp(temp))
24:
25:    case Assign(name, expr):
26:      (code, op) ← flattenIRtoTAC(expr)
27:      exprType ← getType(op)
28:      setType(OpVar(name), exprType)
29:      instr ← TACAssign(OpVar(name), op)
30:      return (code || [instr], OpVar(name))
31:
32:    case other cases: handle recursively...
33:  end match
34: end function
```